
Rapport Blue Prince

Table des matières

1	Architecture Générale	2
2	Gestion de la logique de jeu	4
2.1	États de jeu	4
2.2	Respect des logiques	5
3	Conception du Monde	7
3.1	Les Salles	7
3.2	La Carte	8
4	Gestion des Objets et Inventaires	9
4.1	Comment sont organisé les objets?	9
4.1.1	Pourquoi cette structure?	9
4.1.2	Comment marchent les RoomInventoryObjects?	10
4.2	Comment sont structurés les inventaires?	10
4.2.1	Inventory	10
4.2.2	Room_Inventory	10
4.2.3	Pourquoi cette structure?	11
4.2.4	Fonctionnement des inventaires de chambre	11
4.3	Possibles améliorations	11
5	Génération du pseudo-aléatoire	12
5.1	Le tirage des salles	12
5.2	Le niveau de verrouillage des portes	13
5.3	L’inventaire des salles	13
5.4	L’inventaire des objets de type contenant	13

Introduction

Dans le cadre de l'UE UM4RBT11, il nous est demandé de réaliser un projet de programmation basé sur une architecture orientée objet. Cette année, le thème du projet est de recréer le jeu "BluePrince", un roguelike se déroulant dans un manoir vide. Le but est d'atteindre une chambre se trouvant au fond du manoir nommée "Antechambre". Pour réaliser cela, le joueur peut placer différentes chambres qui lui sont données dans un tirage aléatoire. Chaque chambre comporte des objets divers et variés qui nous aident dans notre objectif. Le long du manoir, le joueur doit résoudre des énigmes afin de réellement accéder à la salle de fin tout en prenant garde à ne pas user tous ses pas (1 pas est consommé à chaque fois qu'on entre dans une chambre). Si le joueur n'a plus de pas, cela signale la fin de la journée. Chaque jour qui passe, le manoir se réinitialise et le joueur doit recommencer son pèlerinage vers l'Antechambre, armé de meilleures connaissances et de plus d'objets.

Dans ce projet, on va recopier les éléments les plus importants du jeu : inventaire du joueur, interaction avec les objets, placement de chambres, navigation du manoir, etc. On n'établira pas de système journalier et l'objectif sera juste d'atteindre l'Antechambre, ce qui comptera comme condition de victoire.

1 Architecture Générale

L'architecture générale a été décidée au tout début du projet avant même de commencer à coder. L'idée était de se mettre d'accord sur une structure, de connaître en avance les modules à coder avant de se répartir équitablement les tâches. De plus, on souhaitait réaliser un code modulable et facilement « scalable ». Ce qui est entendu par là est que si l'on souhaite rajouter plus de fonctionnalités au jeu (nouvelles chambres, nouveaux outils, etc.) on pourrait le faire en utilisant les modules déjà présents dans le code. De plus, l'idée d'utiliser des modules spéciaux qui prennent comme entrée un fichier JSON avec tous les différents objets du jeu et qui renvoient en sortie des instances créées à partir des classes définies dans le code a été envisagée sans être implémentée.

Comment est structuré le projet :

L'architecture du projet comprend 2 modules principaux qui communiquent entre eux :

- L'interface Joueur-Jeu (UI)
- La logique du jeu (Game)

L'utilité de se baser sur une architecture en deux blocs comme celle-ci était de départager l'aspect « front-end » et l'aspect « back-end ».

UI gère toutes les entrées-sorties entre le joueur et le jeu. Si le joueur fournit une action, c'est ce module qui la prend en compte et qui la fournira ensuite au module Game. De même, une fois que Game a géré les inputs du joueur, il donnera comme output une mise à jour de l'état du jeu, qui doit être reflétée sur l'interface graphique pour que le joueur en prenne conscience.

Game gère toute la logique du jeu : le placement des salles et leurs effets, l'utilisation des outils, le mouvement du joueur, etc. Sa logique de fonctionnement dépend des inputs du joueur.

Fichiers concerné :

- `game.py` (Backend)
- `ui.py` (Frontend)
- `main.py` (L'exécutable qui lance le jeu)

Comment Game et UI communiquent entre eux :

La façon dont UI et Game communiquent est à travers un système de MessageBus primitif. Game contient comme attribut un dictionnaire qui détient tous les paramètres pertinents à afficher, comme la position du joueur, la carte des salles, l'inventaire, etc. Ce dictionnaire est fourni à une instance de la classe UI dans le fichier `main.py`. Dans ce même fichier `main`, les inputs de UI sont fournis à Game sous la forme d'une liste d'inputs. La chaîne de communication est la suivante :

1. `main.py` initialise `game_logic = Game()` et `game_interface = UI()`.
2. La boucle principale (`while True`) est dans `main.py`.
3. `ui.run()` récupère les *inputs* bruts (clavier, souris) et les traduit en *commandes* (ex : "UP", "TOGGLE_SETTINGS").
4. `game_logic.handle_inputs(inputs)` reçoit ces commandes et modifie l'état du jeu.
5. `game_logic.publish_data()` prépare un dictionnaire (`self.data`) contenant *uniquement* les informations nécessaires à l'affichage.
6. `game_interface.set_data(data)` reçoit ce dictionnaire et met à jour l'écran.

Les sous modules de UI et Game :

À partir de ces gros modules « front-end » et « back-end », on rajoute des sous-modules à notre guise.

Pour UI, on a décidé de ne pas rajouter de sous-module et de garder un fichier seul qui gère les entrées-sorties et l'affichage de l'état du jeu.

Pour Game, au début du projet, on avait décidé d'instaurer les sous-modules suivants :

1. Player – Gère le joueur, sa position, son mouvement et l'utilisation des outils dans son inventaire.
2. Inventaire – Une classe créant un objet Inventaire qui gère le stockage des outils/objets et les interactions possibles avec ces derniers. Ce module a été utilisé pour le joueur ainsi que pour les chambres.
3. Item – Classe contenant les types d'objets dans le jeu et gérant leur instanciation pour être utilisés dans les autres modules du jeu.
4. Map – Classe contenant l'état de la carte ; elle gère l'addition d'une nouvelle chambre et l'interaction entre le joueur et la carte.

5. `RoomObject` – Classe « chambre » qui gère les paramètres d’une chambre, ses portes, son image, son type et ses effets (en tirage et en entrée).

À cette version, durant le cours du développement, on a rajouté un dernier module :

1. `Random Manager` – Module qui gère toute la logique aléatoire du jeu. Si l’on souhaite avoir plus ou moins de chances d’obtenir une salle dans les tirages ou d’attribuer un inventaire aléatoire à une salle, on peut modifier les paramètres présents dans cette classe.

Possibles améliorations :

- Il aurait été pertinent de modifier la classe `UI` et de la séparer en deux modules : l’un qui gère les entrées-sorties et l’autre qui gère l’affichage, au lieu de tout combiner dans un seul fichier.

2 Gestion de la logique de jeu

Toute la logique du jeu, c’est-à-dire l’application des fonctionnalités implémentées est gérée par `game.py` qui met en commun tous les autres modules pour que tout s’assemble correctement.

La classe `Game` agit comme le cerveau central. Lorsqu’elle est appelée, son constructeur crée les instances principales, entre autres, un `Player()`, une `Map()`, et un `RandomManager()`.

Les informations d’entrées par l’utilisateur sont pré-traitées par l’UI avant d’arriver dans la logique de jeu. En d’autres termes, afin d’améliorer la lisibilité du code, nous avons préféré utiliser les intentions du joueur plutôt que les événements clavier, `Game` ne reçoit pas l’information que Z a été pressée mais plutôt que le joueur souhaite s’orienter vers le nord. Ce sont toutes les méthodes de type `handle` qui gèrent ces intentions en fonction de l’état actuel du jeu.

2.1 États de jeu

Pour ce qui est de la logique du jeu, notre choix a été d’utiliser une variable `self.game_state` dans `game.py` pour dicter le comportement du jeu. On utilise 7 états de jeu différents dont les caractéristiques sont expliquées ci-dessous.

EXPLORING

L’état de jeu principal. Il est actif quand le joueur veut interagir avec la grille de jeu, donc globalement quand il souhaite se déplacer ou s’orienter. Il s’agit du mode par défaut. Dans cet état, les intentions du joueur sont gérées par la méthode `handle_inputs`.

DRAWING_ROOM

Cet état est activé lorsque le joueur se déplace d’une salle vers une case vide. L’exploration est mise en pause. Le jeu attend que le joueur choisisse l’une des trois salles proposées. Les seules commandes acceptées sont la navigation avec les flèches (GAUCHE/DROITE), la confirmation

(ENTRÉE) et la relance du tirage (R pour relancer le tirage des salles). Ces intentions sont gérées par les méthodes `handle_room_selection` et `handle_reroll`.

DOOR.CONFIRMATION

Il s'agit d'un état qui interrompt `EXPLORING` lorsqu'une porte verrouillée est rencontrée. Il attend la confirmation du joueur sur sa volonté de consommer un nombre de clés pour ouvrir la porte. Le jeu est en attente d'une confirmation ou d'une annulation, et les inputs sont temporairement limités à ce choix et gérés par `handle_door_confirmation`.

ACTION.CONFIRMATION

Similaire à `DOOR.CONFIRMATION`, mais cet état est déclenché par une action dans une salle (comme "Ouvrir un coffre" ou "Utiliser un marteau"). Il demande une confirmation avant de consommer un objet ou de valider l'action. Les entrées sont gérées par `handle_confirmation`.

SETTINGS

Un état de pause qui affiche le menu des paramètres par-dessus le jeu. Les inputs, gérées par `handle_inputs`, sont ré-assignés au contrôle des sliders de volume, des cases à cocher, et des boutons "Redémarrer" ou "Quitter".

VICTORY

Un état terminal activé lorsque le joueur atteint la salle `AnteChamber`. Le jeu est terminé et gagné. L'UI affiche un écran de victoire et la plupart des inputs sont désactivés conservant uniquement les entrées qui concernent les réglages.

GAME.OVER

L'autre état terminal, conçu pour s'activer lorsque le nombre de pas du joueur est à 0. Dans l'implémentation actuelle, il est signalé par un `warning_message`, mais il est géré comme un état final où les inputs sont également bloqués, mettant fin à la partie.

Cette partition en état de jeu permet d'éviter les confusions dans les commandes et les bugs associés mais rend aussi le code beaucoup structuré et clair.

2.2 Respect des logiques

Au-delà du simple routage d'inputs, `game.py` veille à ce que le respect des logiques implémentées dans les autres modules règne.

Mouvement du joueur

Tout d'abord, le mouvement du joueur. Il est finement calculé afin d'assurer la cohérence de jeu. L'algorithme suivant fait état du fonctionnement de cette méthode.

1. Demander à la salle actuelle si elle a une sortie dans la direction dans laquelle le joueur souhaite poursuivre son aventure.
2. Calculer la position cible et demander à **Map** ce qui s'y trouve.
3. Si une porte existe, demander à la salle son niveau de verrouillage
4. Demander à l'inventaire du joueur (**Player**) s'il possède des ressources.
5. Si la porte est verrouillée et que le joueur a les clés, on change l'état de jeu en **DOOR_CONFIRMATION** et on stocke l'action en attente.
6. Si la case cible est vide, on change l'état de jeu en **DRAWING_ROOM** et on peut appeler le tirage des salles.

Sélection de salle

De la même façon que pour le déplacement du joueur, le choix de la prochaine salle est soumis à des contraintes que les méthode de **Game** force le joueur à respecter. Le processus se déroule en plusieurs étapes :

1) Rotation intelligente

Au moment où le joueur entre dans une case vide, le jeu ne se contente pas de tirer 3 salles. Il appelle immédiatement `self.find_best_rotation()` pour chacune des 3 salles proposées. Les salles sont donc présentées au joueur déjà dans leur meilleure orientation possible.

Cette méthode est au coeur de l'expérience utilisateur. Quoi de plus frustrant que des salles qui n'orientent pas vers l'objectif final ! c'est pourquoi nous avons choisi d'appliquer heuristique qui favorise la progression vers l'objectif. Nous avons décidé de ne pas proposer au joueur la première rotation valide que le programme calcul mais bien la meilleure. Nous définissons la meilleure rotation comme étant celle qui donne une porte vers le nord, si une telle rotation existe alors inutile de chercher plus longtemps. En revanche, dans de nombreux cas, cette rotation n'existe pas, on définit alors la "moins pire" des rotations comme étant celle qui n'a PAS de sortie vers le sud, l'objectif étant d'éviter les motifs d'escargot qui bloque rapidement le joueur.

2) Confirmation de la sélection

Une fois que le joueur valide son choix (parmi les 3 salles pré-orientées), cette méthode orchestre l'application de la logique :

1. Elle vérifie les ressources du joueur.
2. Si le joueur peut payer, elle consomme les ressources (pas et/ou diamants).
3. Elle déclenche l'éventuel effet au tirage de la salle.
4. Elle demande au **RandomManager** d'assigner un inventaire d'actions à la salle.
5. Elle place la salle sur la **Map**.
6. Elle déplace le **Player** dans la nouvelle salle.

7. Enfin, elle déclenche l'éventuel effet à l'entrée de la salle.

3 Conception du Monde

3.1 Les Salles

Pour la conception du monde, notre choix d'architecture principal a été d'utiliser l'héritage. Nous avons défini une classe de base `RoomObject` dans le fichier `room.py`. Chaque salle spécifique du jeu, soit près d'une cinquantaine au total, est implémentée comme une classe distincte qui hérite de `RoomObject`, par exemple pour la salle `Bedroom`, on a : `class Bedroom(RoomObject)`.

Cela nous permet de rendre notre projet plus facilement modifiable et d'ajouter des salles comme on le souhaite sans effectuer de changements fastidieux. Pour ajouter une nouvelle salle, il nous suffit de créer sa classe dans `room.py` en y définissant ses attributs et ses effets uniques, puis on l'ajoute à la liste `full_room_deck` dans `random_manager.py`. Le reste du code, notamment `game.py`, n'a pas besoin d'être modifié.

Toute la logique d'une salle (son coût, sa rareté, ses portes, ses effets, son type et ses conditions de placement) est contenue dans sa propre classe, ce qui rend le code plus propre et facile à maintenir.

Attributs de la classe de base

La classe `RoomObject` définit les attributs communs à toutes les salles qui sont ensuite utilisés par les autres modules :

- **base_exits et orientation** : Pour économiser les ressources, nous n'avons pas créé plusieurs images par salle (correspondant aux rotations possibles) car nous faisons tourner directement dans le code l'image de la salle pour une rotation. Nous stockons un attribut `base_exits` correspondant aux directions des portes de la salle (ex : `[0, 2]` pour Nord et Sud). Ensuite la méthode `has_exits(direction)` calcule les sorties réelles en fonction de l'attribut `orientation` (un entier de 0 à 3).
- **rarity et cost** : Ces attributs de classe sont lus par le `RandomManager` pour gérer le tirage pondéré et le coût en gemmes.
- **room_type** : Un attribut indiquant dans quelle couleur de salles on se situe (conformément au Wiki de Blue Prince), par exemple : `'Green Room'`, `'Bedroom'` ou `'Hallway'`, ces types de salles permettent d'être utilisés par les effets spéciaux d'autres salles (comme `Terrace` qui affecte les `'Green Rooms'` ou `Nursery` qui affecte les `'Bedrooms'`).
- **placement_constraints** : Un attribut qui permet de définir des règles de placement, lues par `RandomManager`. Par exemple, `West_Wing_Hall` ne peut apparaître que sur les colonnes ouest de la carte ou `Cloister` ne peut apparaître que au milieu de la map.

Gestion des effets spéciaux

Pour gérer les effets spéciaux, nous avons défini deux méthodes de base, `on_draft` et `on_entry`, dans la classe `RoomObject`. Ensuite, chaque salle redéfinit ces méthodes pour y mettre sa propre logique, ainsi le fichier `game.py` n'a pas besoin de savoir quel type de salle il gère, il lui suffit d'appeler `salle.on_entry()` et `salle.on_draft()` et l'effet spécial défini (s'il y en a un) s'exécute automatiquement :

- **`on_draft(game_logic)`** : Cette méthode est appelée par `game.py` une seule fois au moment où le joueur sélectionne la pièce, elle est utilisée pour les effets permanents ou les bonus immédiats.
- **`on_entry(game_logic)`** : Cette méthode est appelée par `game.py` chaque fois que le joueur entre dans la salle, elle est utilisée pour les effets répétables.

Grâce à ce système, nous avons implémenté différents effets spéciaux tel que :

- **Gain/Perte de ressources à l'entrée** : Comme `Bedroom` (+2 Pas) ou `Chapel` (-1 Pièce).
- **Gain/Perte de ressources au tirage** : Comme `Guest_Bedroom` (+10 Pas) ou `Weight_Room` (-50% des Pas).
- **Modification des probabilités de tirage** : Comme `Furnace` (augmente la chance des 'Red Rooms') ou `Greenhouse` (augmente la chance des 'Green Rooms').
- **Modification des probabilités d'objets** : Comme `Veranda` (plus d'objets dans les 'Green Rooms') ou `Maids_Chamber` (moins d'objets partout).
- **Dispersion d'objets sur la carte** : Comme `Office` (disperse des Pièces) , `Patio` (disperse des Gemmes) ou `Locker_Room` (disperse des Clés).
- **Modification de la pioche de salles** : Comme `Chamber_of_Mirrors` (active les doublons) ou `The_Pool` (ajoute 3 nouvelles salles à la pioche).
- **Modification de l'état global du jeu** : Comme `Terrace` (rend toutes les 'Green Rooms' gratuites) ou `Foyer` (déverrouille tous les 'Hallways' passés et futurs).
- **Activation de bonus futurs** : Comme `Nursery` (chaque 'Bedroom' future donne +5 Pas) ou `Her_Ladyships_Chamber` (active un bonus lors de la prochaine visite du Boudoir).
- **Interaction unique en salle** : Comme la `Rotunda` qui permet au joueur d'appuyer sur 'T' pour faire pivoter la salle.

3.2 La Carte

La carte est définie par la Classe `Map`. Celle ci contient un attribut `mapping`, représentant la carte par un array numpy de taille 5x9. Cette structure permet de rajouter des Chambres sur chaque index de l'array. Deux des indexes sont fixes, (2,8) qui représente le hall d'entrée et (2,0) qui représente l'Antechambre. Cette structure est très simple et pourtant très puissante dû à la rapidité des numpy arrays et à la facilité de manipulation de ces derniers.

4 Gestion des Objets et Inventaires

L'inventaire et les objets sont intrinsèquement liés. On distingue deux types d'inventaires :

- **Inventory** — Inventaire général utilisé pour le joueur et pour les objets contenant d'autres objets (comme des coffres ou des trous).
- **RoomInventory** — Inventaire d'une salle, contenant plusieurs objets.

Pour les objets, on a également deux catégories :

- Les objets généraux (**Item**)
- Les objets appartenant à des salles (**RoomInventoryObject**)

4.1 Comment sont organisé les objets ?

On a plusieurs types d'objets dans le jeu : les objets consommables, les objets non consommables, et les objets qui redonnent des pas (que l'on appelle régénératifs). On a créé une classe abstraite appelée **Item**, qui détient les méthodes et attributs communs à chaque classe.

Attributs

- **Name** – Le nom de l'objet, utilisé pour l'affichage des messages dans l'UI ainsi que comme référence à l'objet.
- **Description** – Une description de l'objet affichée sur l'UI.
- **Image_path** – L'image utilisée pour l'objet s'il y en a une (ceci est utilisé pour les objets présents dans l'inventaire du joueur ; les objets régénératifs qui régénèrent une quantité d'un objet n'ont pas besoin d'image car ils ne sont jamais ajoutés à l'inventaire).
- **Quantity** – La quantité de l'objet.

Méthodes

- **Set_quantity** – Fonction pour définir la quantité de l'objet.
- **Use(self, player, amount)** – Méthode abstraite : elle utilise l'objet depuis lequel on l'appelle. Chaque méthode **Use()** renvoie un booléen pour indiquer si l'action a été réalisée ou non.
- **Return_item_with_amount(self, quantity)** – Méthode non abstraite présente dans toutes les classes héritant de **Item**. Elle renvoie une copie de l'objet avec une quantité sélectionnée par l'utilisateur. Utile pour la création d'inventaires aléatoires.

4.1.1 Pourquoi cette structure ?

La force de cette structure vient de ses méthodes :

- **Return_item_with_amount** – On peut définir tous nos objets de base (ex. pommes, bananes, pelle, dés, etc.) dans **item.py**. Quand on souhaite avoir un objet dans une salle,

un coffre, etc., on peut appeler cette fonction sur un objet instancié pour obtenir une copie avec une quantité désirée. Cela permet de référencer un type d'objet sans avoir à le redéfinir.

- **Use** étant abstraite, on doit la spécifier pour chaque classe. Ainsi, si on souhaite utiliser un objet (ex. utiliser une pelle pour creuser un trou, ou utiliser un dé pour refaire un tirage de salle), il suffit d'appeler `Use()`. Par exemple : si l'action « relancer un tirage » utilise l'objet `Dice` de quantité 1, alors appeler `Dice.use()` va automatiquement soustraire 1 dé de l'inventaire du joueur, si ce dernier en possède assez. Sinon `Use()` renvoie `False`, ce qui informe le programme de la suite à donner.

4.1.2 Comment marchent les `RoomInventoryObjects` ?

Dans le cas où l'on a des objets dans des chambres avec lesquels on peut interagir (les récolter, les ouvrir, les creuser, les acheter, etc.), il faut un système de messages à envoyer à l'interface graphique. De plus, si l'on souhaite rajouter une « condition d'interaction » (la condition à remplir pour interagir avec l'objet), il faut ajouter un attribut qui gère cela.

Ces fonctionnalités auraient pu être intégrées directement dans la classe `Item`. Cependant, comme le projet avait déjà bien avancé, il aurait été problématique de modifier le code existant. On a donc choisi de créer un *wrapper* : la classe `RoomInventoryObject` détient une instance de `Item`, ainsi que les différents messages et la condition d'activation.

4.2 Comment sont structurés les inventaires ?

Comme dit plus tôt, on a deux types d'inventaires : `Inventory` (inventaire général) et `Room_Inventory` (inventaire de chambre).

4.2.1 `Inventory`

Attributs

- **Inventory** – Dictionnaire ayant comme clé le nom du type d'objet (ex. « Apple », « Banana », etc.) et comme valeur une instance des classes d'objets.

Méthodes

- `Add_item(self, item_to_add)` – Ajoute un item à l'inventaire.
- `Add_inventory(self, other)` – Ajoute un inventaire complet à l'inventaire.
- `Get_quantity(self, item_name)` – Renvoie la quantité du type d'objet demandé.
- `Get_all_items(self)` – Renvoie tous les objets détenus dans l'inventaire.

4.2.2 `Room_Inventory`

Attributs

- **Inventory** – Liste contenant les `RoomInventoryObjects` présents dans la salle.

Méthodes

- `addInventory(self, inventory_Item)` – Ajoute un objet de type `RoomInventoryObject`.
- `get_action_number(self)` – Renvoie le nombre d’actions disponibles dans la salle.
- `get_action_messages(self)` – Renvoie les messages d’interaction (action réalisable, réussie, ratée).
- `checkInv_activation_condition(self, player, room_object)` – Vérifie si le joueur remplit la condition d’activation.
- `use_item(self, player, room_object)` – Interagit avec l’objet et applique ses effets sur le joueur.
- `Set_inventory(self, inventory)` – Définit une nouvelle liste d’objets.
- `Return_inventory_copy(self)` – Renvoie une copie de l’instance.
- `Handle_action(self, player, action_index)` – Gère les inputs du joueur pour effectuer une action.

4.2.3 Pourquoi cette structure ?

Cette structure est née du compromis établi plus tôt avec les objets et la création du wrapper `RoomInventoryObject`. Si la structure était à refaire, on généraliserait le tout, mais des efforts ont été faits pour que l’opération reste logique.

4.2.4 Fonctionnement des inventaires de chambre

- Une chambre reçoit un inventaire via le *random manager*.
- L’inventaire de la chambre se voit ajouter des `RoomInventoryObjects`.
- Il existe deux types de `RoomInventoryObjects` : ceux qui détiennent un `Item`, et ceux qui détiennent un `Inventory`.
- Si un `RoomInventoryObject` détient un `Item`, l’objet est directement interactif pour le joueur (ex. une pomme). Dès que la condition d’activation est vérifiée, l’objet est utilisé.
- Si un `RoomInventoryObject` détient un `Inventory`, alors il sert de conteneur. Si le joueur vérifie la condition d’utilisation, tous les objets du coffre sont ajoutés à l’inventaire de la chambre. Exemple : si un trou contient 2 pommes et 1 clé, l’inventaire de la chambre gagne `Apple x2` et `Key x1`.

4.3 Possibles améliorations

1. Réécrire le code pour mettre les attributs et méthodes de `RoomInventoryObject` dans les classes héritant de `Item`.
2. Créer une nouvelle classe `Inventory_Item` héritant de `Item`, détenant un inventaire et localisant la logique de traitement dans la méthode `Use()`.
3. Nettoyer la logique de gestion des inputs dans `Handle_action`, qui contient probablement des éléments redondants dus à d’anciennes versions du code.

5 Génération du pseudo-aléatoire

Nous avons centralisé la gestion de l'aléatoire dans un module `random_manager.py`. L'objectif était de créer notre propre module `random`, donc, tout naturellement, nous lui avons consacré à l'aléatoire son propre module.

Nous avons fait le choix de définir certains dictionnaires réglant la difficulté du jeu en tant que constantes de configuration en dehors de la classe afin de faciliter le changement de difficulté du jeu. De plus, cette décision nous permet aussi de grandement simplifier les phases de test du jeu de notre côté et par conséquent les phases d'équilibrages aussi. Pour finir, la création d'un module à part entière améliore aussi grandement la lisibilité et la compréhension du module `game.py` qui n'a pas à se soucier de la rareté ou des probabilités. Il demande juste 3 salles, et le manager s'occupe de la complexité.

Dans la classe `RandomManager`, nous distinguons 4 implémentations d'aléatoires différents :

1. Le tirage des salles
2. Le niveau de verrouillage des portes
3. L'inventaire des salles
4. L'inventaire des objets de type contenant

5.1 Le tirage des salles

Pour ce qui est du tirage des salles, il est géré par `draw_placable_rooms`. Cette méthode est plus complexe qu'un simple tirage aléatoire. Elle assure que les 3 salles proposées au joueur sont non seulement valides mais aussi pertinentes.

Filtrage des salles plaçables

Avant de tirer au sort, la méthode parcourt l'intégralité de la pioche de salles, soit `current_room_deck`, soit `full_room_deck` si l'effet `allow_duplicates` est actif. Pour chaque salle, elle appelle `is_room_placable`, méthode décrite dans la section suivante.

Validation de la plaçabilité (`is_room_placable`)

Cette méthode est cruciale. Elle teste la salle candidate pour chacune des 4 rotations possibles. Une salle est jugée plaçable si une de ses rotations respecte ces conditions :

- Respecter la direction d'entrée du joueur (`direction_of_entry`).
- Respecter les contraintes de placement (west et east corridor)
- Compatibilité avec ses voisins qui est vérifiée par `map.is_placement_valid()`.

Ainsi, les salles qui passent ce test forment le deck dans lequel on tire les 3 salles parmi lesquelles choisir.

Tirage pondéré et garantie

Une fois la liste des salles plaçables obtenue, la méthode effectue un tirage pondéré en tenant compte de la rareté de chacune des salles recensée dans le dictionnaire `RARITY_WEIGHTS` ainsi que les éventuelles multiplicateurs de probabilité provoqués par d'autres effets.

La dernière condition est de garantir qu'au moins une salle parmi les 3 a un coût en gemme nul. Pour ce faire, on crée simplement une liste des salles gratuites contenues dans le deck des salles plaçables et on en tire une dedans puis 2 autres dans le deck d'origine.

5.2 Le niveau de verrouillage des portes

L'aléatoire du verrouillage est géré par `assign_locks_to_room` et `calculate_lock_level`. L'objectif est de créer une courbe de difficulté progressive. En effet, le niveau de verrouillage augmente en même temps que l'index de la ligne sur laquelle le joueur se trouve.

On fixe d'abord les niveaux de verrouillage des portes pour toutes les salles de la première et de la dernière ligne de la grille de jeu, pour lesquelles on souhaite respectivement un niveau de 0 et 2. Pour les autres lignes, on augmente simplement les probabilités des verrouillages de portes élevés que l'on stocke dans un dictionnaire `LOCK_PROB`.

Cette logique n'est outrepassée que dans le cas où il s'agit d'un corridor ou si un autre effet de salle entre compte (Foyer).

5.3 L'inventaire des salles

La génération des actions (objets à ramasser, coffres, trous, casiers) dans une salle est gérée par `assign_inventories_to_room`.

De base, la probabilité de trouver des objets apparaissant dans une salle est fixée à 60% puis des modifications sous forme de multiplicateurs sont appliquées en fonction des effets ou des objets possédés par le joueur. Par la suite, le nombre d'objet à faire apparaître est choisi au hasard (pondéré) entre 1 et 3 objets et peut être modifié si le joueur possède un certain type d'objet. Une fois ces quantités définies, on peut tirer les actions qu'on offre au joueur dans la liste des objets disponibles avec leur pods associés.

Si le joueur possède un charme chroma alors la probabilité d'apparition d'objets est augmentée, s'il possède un détecteur de métal, la probabilité que les objets qui apparaissent soient métalliques est augmentée.

5.4 L'inventaire des objets de type contenants

Enfin, pour l'assignation de des inventaires des contenants (coffres, casiers, trous), nous avons choisi de programmer leur propre logique, différente de celui des inventaires des salles.

Lorsque l'action tirée à l'étape précédente est un "Chest", "Hole" ou "Locker", la méthode `generate_random_loot_inventory` est appelée pour remplir ce contenant.

On définit donc des listes d'objets apparaissables différentes pour les coffres/casiers et les trous (on ne voudrait pas trouver une pelle dans un trou alors que la pelle est nécessaire pour creuser ce trou ...). Ensuite on récupère un peu la logique expliquée dans la section précédente pour définir le nombre d'objets contenu dans le contenant en tenant compte du fait que les coffres ne peuvent pas être vides mais les casiers et trous peuvent l'être.

Nous avons aussi implémenté une logique anti-doublon qui vérifie si l'objet est une instance de la classe des objets non consommables et si c'est le cas et que le joueur a déjà cet objet dans son inventaire, alors on ne l'ajoute pas dans les objets qui seront contenus dans le coffre. Cette logique pose néanmoins un problème, si le seul objet qui a été choisi était un objet non consommable que le joueur possède déjà alors le coffre se retrouve vide ! Pour éviter cette situation, on rajoute une logique de secours qui rajoute au moins 1 objet dans le coffre si le coffre est vide.

Conclusion

Ce projet nous a permis de mettre en œuvre les concepts de programmation orientée objet pour développer un jeu fonctionnel, nous avons pour cela fait les choix suivants afin de structurer au mieux notre code :

Premièrement, on a séparé la logique de l'affichage (une approche similaire à MVC), en isolant la logique dans `game.py` et la présentation dans `ui.py`. La communication entre ces deux modules, gérée par `main.py`, assure un code découplé et facile à maintenir.

Deuxièmement, l'implémentation d'une machine à états finis dans `game.py` (avec des états comme `EXPLORING` ou `DRAWING_ROOM`) nous a permis de gérer la complexité des commandes utilisateur et d'éviter les bugs d'entrées contradictoires en fonction du contexte.

Troisièmement, nous avons utilisé l'héritage pour concevoir le gros du jeu, en définissant des classes de base `RoomObject` et `Item`, nous avons pu créer de nombreuses salles et des objets uniques simplement en étendant ces classes.

Enfin, nous avons implémenté toute la logique pseudo-aléatoire dans le module `Random_Manager`, ce qui a permis de simplifier `game.py`, qui peut se contenter de "demander" 3 salles sans se soucier des probabilités, des poids, ou des effets de rareté.

Points Forts

Ces choix pour l'architecture permettent à notre code d'avoir :

- **Modularité** : Chaque module a une responsabilité unique, la logique, l'affichage et l'aléatoire. Cela nous a permis de travailler plus facilement en équipe notamment pour le

débogage.

- **Extensibilité** : C'est le plus gros avantage de notre code, pour ajouter une nouvelle salle, il a suffi de créer une nouvelle classe dans `room.py` et de l'ajouter à la liste `full_room_deck`. Le reste du code, y compris les effets spéciaux, s'adapte automatiquement.
- **Testabilité** : En isolant la logique, comme `calculate_lock_level` dans `Random_Manager`, nous avons pu tester et équilibrer des parties spécifiques du jeu sans avoir à lancer l'interface graphique complète.

Pistes d'Amélioration

Nous avons tout de même identifié plusieurs axes d'amélioration pour une future version :

- **Optimisation du cache d'images** : Nous avons implémenté un `room_image_cache` pour gérer les rotations d'images à la volée. On pourrait étendre ce système pour pré-charger toutes les images au démarrage du jeu afin d'éliminer toute latence lors de l'affichage de nouvelles salles.